

# ACA Mini-Project

Design A vs Design B vs Design C — Architecture, Cycles, Requests, Stalls and Synthesis Results

## 1. Final Synthesis Comparison

| Metric       | Design A<br>Parallel | Design B<br>K-serial | Design C<br>K-serial + Double Buffer |
|--------------|----------------------|----------------------|--------------------------------------|
| Cells        | 113,160              | 62,151               | 113,160                              |
| Logic depth  | 71                   | 57                   | 71                                   |
| Area × delay | 8,034,360            | 3,542,607            | 8,034,360                            |
| W1 cycles    | 40,961               | 53,249               | 41,729                               |
| W1 stalls    | 1                    | 12,289               | 769                                  |
| W1 errors    | 0                    | 0                    | 0                                    |
| W2 cycles    | 299,009              | 397,313              | 299,777                              |
| W2 stalls    | 1                    | 98,305               | 769                                  |
| W2 errors    | 0                    | 0                    | 0                                    |

**Result:** All three designs passed W1 and W2 with zero errors. Design C uses double buffering to obtain near-reference throughput while retaining K-serial computation.

## 2. Common Specification

**Operation:**  $C[4][4] += A[4][4] \times B[4][4]$ . A and B are signed 8-bit values; C is a signed 32-bit accumulator.

**Per 4×4 tile:** 64 MAC operations. The interface needs 8 load requests (4 A rows + 4 B rows) plus 1 Mul request = **9 requests/tile**.

| Workload | Output tiles | K-tiles / output tile | Mul operations | Requests |
|----------|--------------|-----------------------|----------------|----------|
| W1       | 256          | 16                    | 4,096          | 40,960   |
| W2       | 256          | 128                   | 32,768         | 299,008  |

### 3. Design A — Fully Parallel Reference

Design A calculates all four K products for every  $C[i][j]$  in one Mul cycle. There are 16 output elements and 4 products per output, giving **64 multipliers** operating in parallel.

**Mul latency:** 1 computation cycle.

**Hardware:** 64 multipliers + 16 four-input accumulation trees.

**W1:** 40,960 requests, 40,961 cycles, 1 stall, 0 errors.

**W2:** 299,008 requests, 299,009 cycles, 1 stall, 0 errors.

**Synthesis:** 113,160 cells; logic depth 71; area×delay 8,034,360.

**Advantage:** minimum computation latency. **Disadvantage:** highest parallel arithmetic cost.

### 4. Design B — K-Serial

Design B serializes the K dimension while keeping all 16 output elements parallel. Therefore only **16 multipliers** are needed.

**Mul sequence:** K=0, K=1, K=2, K=3 → 4 computation cycles per Mul.

Because the DUT is busy for those four cycles, the next A/B tile cannot be loaded during computation. Each Mul therefore adds 3 cycles compared with Design A. W1 has 4,096 Mul operations, giving 12,288 extra cycles. W2 has 32,768 Mul operations, giving 98,304 extra cycles.

**Synthesis:** 62,151 cells; logic depth 57; area×delay 3,542,607.

**W1:** 53,249 cycles, 12,289 stalls, 0 errors.

**W2:** 397,313 cycles, 98,305 stalls, 0 errors.

**Trade-off:** much lower hardware cost, but lower throughput.

## 5. Design C — K-Serial with Double Buffering

Design C keeps the 16-multiplier K-serial engine from Design B but adds two A/B storage banks: **Bank 0 = A0/B0** and **Bank 1 = A1/B1**.

The active bank is used for multiplication while the inactive bank receives the next tile. After K=3 completes, the active-bank bit flips. This allows the next tile to be ready while the current tile is still being computed.

**Compute sequence:** K=0, K=1, K=2, K=3 → 4 computation cycles per Mul.

**Overlap:** current bank computes while the other bank loads A/B.

**W1:** 40,960 requests → 41,729 cycles → 769 stalls → 0 errors.

**W2:** 299,008 requests → 299,777 cycles → 769 stalls → 0 errors.

For W2, Design C needs only **768 cycles more than Design A**, while Design B needs 98,304 additional cycles. This demonstrates that double buffering hides most of the K-serial computation latency.

**Synthesis:** 113,160 cells; logic depth 71; area×delay 8,034,360 in the current implementation.

## 6. Why the Stall Counts Differ

**Design A:** Mul completes in one cycle, so only 1 measured stall occurs in each workload.

**Design B:** each Mul occupies four cycles and blocks the next tile's loads. The three additional cycles per Mul accumulate across all K-tiles.

**Design C:** the inactive bank loads while the active bank computes. Most of the K-serial latency is therefore hidden, leaving mainly startup, drain and boundary effects.

## 7. Direct Design Trade-off

| Feature          | Design A   | Design B | Design C           |
|------------------|------------|----------|--------------------|
| Multipliers      | 64         | 16       | 16                 |
| A/B banks        | 1          | 1        | 2                  |
| Mul computation  | 1 cycle    | 4 cycles | 4 cycles           |
| Load during Mul? | Not needed | No       | Yes, inactive bank |
| W1 cycles        | 40,961     | 53,249   | 41,729             |
| W2 cycles        | 299,009    | 397,313  | 299,777            |
| Cells            | 113,160    | 62,151   | 113,160            |
| Logic depth      | 71         | 57       | 71                 |

## 8. Final Conclusion

Design A uses maximum parallelism to minimize latency. Design B trades latency for area by serializing K. Design C uses double buffering to hide the serialization latency by overlapping computation with loading. All three designs passed the supplied W1 and W2 workloads with zero errors.

**Viva summary:** A uses parallelism; B trades latency for area; C uses double buffering to recover throughput.